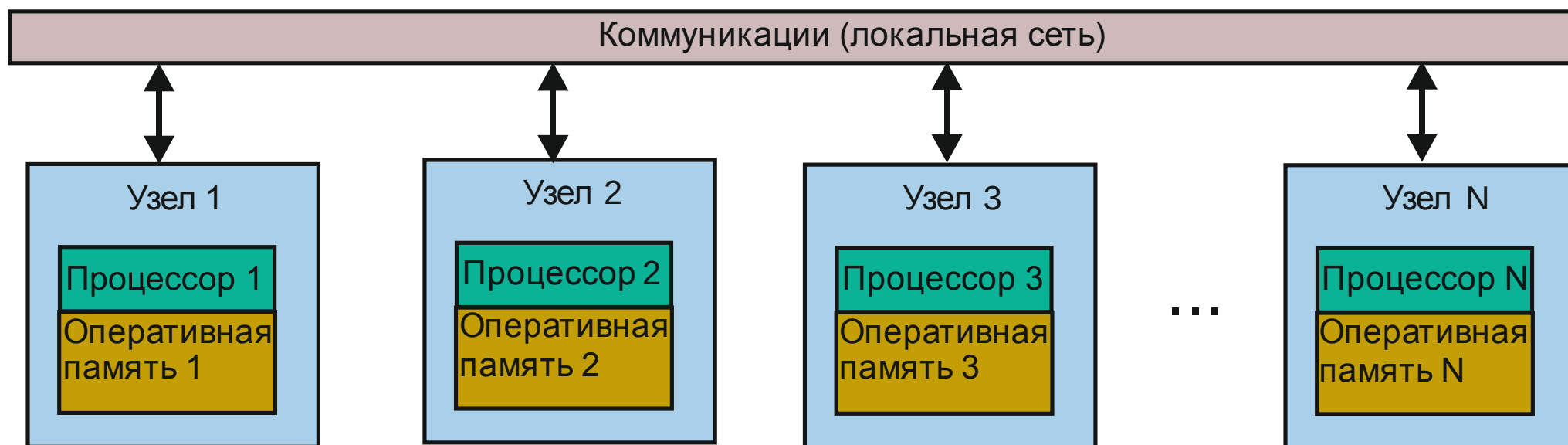


ПАРАЛЛЕЛЬНОЕ ПРОГРАММИРОВАНИЕ С ИСПОЛЬЗОВАНИЕМ ТЕХНОЛОГИИ MPI

- Технология MPI наиболее эффективна на кластерных системах, представляющих собой узлы (компьютеры, системные блоки) с (примерно) одинаковыми характеристиками, соединенные быстрой локальной сетью, и функционирующие под управлением одной и той же ОС.



- MPI (Message Parsing Interface, «Программирование на основе передачи сообщений») представляет общепринятый стандарт для организации программ в форме параллельно работающих процессов на вычислительных системах с распределенной памятью
- Модель программы MPI может быть охарактеризована как «одна программа – множество процессов» («single program – multiple processes»)
- MPI в значительной степени снижает остроту проблемы переносимости программного обеспечения между различными параллельными вычислительными системами с распределенной памятью. Параллельная программа, разработанная на языках С или Fortran с использованием библиотек MPI, как правило, будет работать на различных вычислительных платформах
- MPI содействует эффективности параллельных вычислений, т.к. для всех типов параллельных вычислительных систем существуют реализации MPI, обеспечивающие наилучшее использование аппаратного обеспечения
- MPI уменьшает сложность разработки параллельных программ, т.к., с одной стороны, основные операции передачи данных предусмотрены стандартом MPI, а с другой стороны, имеется большое количество программного обеспечения, использующего стандарт MPI.

Параллельные программы MPI

- Под параллельной MPI-программой понимается множество одновременно выполняемых процессов. Процессы могут выполняться на разных процессорах.
- Каждый процесс параллельной программы порождается на основе одного и того же программного кода. Данный программный код (бинарный загрузочный модуль и разделяемые библиотеки) должен быть доступен на всех используемых процессорах.
- Количество процессов параллельной MPI-программы и (без редко используемых специальных средств последних стандартов) не может изменяться.
- Компиляция и сборка MPI-программ на UNIX-подобных ОС реализуется командами `mpicc` и `mpicxx`, аналогичными командам вызова системных компиляторов C/C++
- Как правило, запуск параллельной программы в большинстве реализаций MPI осуществляется командой
 - `mpiexec -n <число_процессов> <исполняемая_программа>`
- Все процессы MPI-программы последовательно пронумерованы от 0 до $p - 1$, где p – число процессов. Номер процесса называется его рангом.

- Основу MPI составляют операции передачи сообщений. Различаются **парные** (point-to-point) операции между двумя процессами и **коллективные** (collective) коммуникационные действия для одновременного взаимодействия нескольких процессов.
- Для выполнения парных операций используются различные режимы: синхронный, блокирующий и т.д.
- Процессы параллельной MPI-программы объединяются в **группы**.
- Под коммуникатором в MPI понимается специально создаваемый служебный объект, который объединяет в своем составе группу процессов и ряд дополнительных параметров (контекст), используемых при передаче данных.
- Парные операции выполняются лишь для процессов, принадлежащих одному и тому же коммуникатору.
- Коллективные операции выполняются для всех процессов одного коммуникатора.
- Как результат, указание используемого коммуникатора является обязательным для операций передачи данных в MPI.
- Все имеющиеся в программе процессы входят в состав создаваемого по умолчанию коммуникатора с идентификатором **MPI_COMM_WORLD**.

- В ходе выполнения MPI-программы могут создаваться новые и удаляться существующие группы процессов и коммутаторы.
- Один и тот же процесс может принадлежать разным группам и коммутаторам.
- При выполнении операций передачи сообщений в функциях MPI требуется указывать тип пересылаемых данных.
- MPI содержит **большой набор базовых типов данных**, совпадающих с типами данных C/C++
- Имеется возможность создания производных типов данных, которые используются в передаваемых сообщениях
- По умолчанию, логическая топология линий связи между процессами имеет структуру топологии полного графа вне зависимости от наличия реальных физических каналов связи между процессорами
- Имеется возможность представления множества процессов в виде решетки процессов произвольной размерности либо в виде решетки-тора произвольной размерности
- Также имеются средства формирования топологий любого требуемого типа

Инициализация и завершение MPI-программ

- Прототипы функций MPI в стандартном заголовочном файле `mpi.h`
- В случае успешного завершения функции MPI возвращают `MPI_SUCCESS`
- При начале параллельного выполнения MPI-программ первой вызываемой функцией MPI должна быть функция

`int MPI_Init(int *argc, char** argv)`

где `argc` — указатель на количество параметров командной строки, `argv` — параметры командной строки. (Значениям по умолчанию соответствует задание параметров `NULL, NULL`)

- Последней вызываемой функцией MPI обязательно должна являться функция

`int MPI_Finalize(void)`

- В каждом процессе MPI вызов этих функций должен выполняться лишь по одному разу.
- Перед вызовом `MPI_Init` может быть выполнена функция

`int MPI_Initialized(int *flag)`

возвращающая в переменной `flag` значение `true`, если инициализация MPI уже была выполнена. Перед вызовом `MPI_Finalize` может быть выполнена функция

`int MPI_Finalized(int *flag)`

с аналогичным предназначением

Определение количества и ранга процессов

- Определение количества процессов параллельной MPI-программы, входящих в некоторый коммуникатор, определяется при помощи функции

`int MPI_Comm_size(MPI_Comm Comm, int *Size)`

где `Comm` – коммуникатор, размер которого определяется,

`Size` – определяемое количество процессов в коммуникаторе

- Для определения ранга процесса используется функция

`int MPI_Comm_Rank(MPI_Comm Comm, int *Rank)`

где `Comm` – коммуникатор, размер которого определяется,

`Rank` – номер, т.е. ранг процесса в коммуникаторе

Переменная `Rank` получит значение от 0 до `Size-1`, индивидуальное для каждого из параллельно работающих процессов.

Пример простейшей MPI-программы

```
//MPI_Simpliest_c_Bind.cpp
#include "mpi.h"
#include <iostream>

using namespace std;
int main(void)
{int NProc, ProcId;
  MPI_Init(NULL, NULL);
  MPI_Comm_size(MPI_COMM_WORLD, &NProc);
  MPI_Comm_rank(MPI_COMM_WORLD, &ProcId);
  cout<<"Total number of processes: "<<NProc
    <<"  Process identifier: "<<ProcId<<endl;
  MPI_Finalize();
  return 0;
}
```

ПАРНЫЕ МЕЖПРОЦЕССНЫЕ ОБМЕНЫ

Операции блокирующей передачи и приема сообщений

- Блокирующая передача сообщений реализуется функцией
`int MPI_Send(void* buf, int count, MPI_Datatype datatype,
int dest, int tag, MPI_Comm comm)`

Здесь

`buf` – начальный адрес буфера в оперативной памяти, в котором располагаются данные отправляемого сообщения;

`count` – число элементов в буфере отправки

`datatype` – тип данных каждого элемента в буфере передачи

`dest` – ранг процесса-получателя

`tag` – значение-тэг (от 0 до 32767), используемое для идентификации сообщения

`comm` – коммуникатор, в рамках которого выполняется передача сообщения.

Предопределенный коммуникатор `MPI_COMM_WORLD` разрешает обмен для всех процессов, которые доступны после инициализации MPI. Блокировка гарантирует корректность использования всех переменных после возврата из данной процедуры. В случае успешного завершения возвращает предопределенное значение `MPI_SUCCESS`

Соответствие типов данных MPI и C/C++

Тип данных MPI	Тип данных C/C++
MPI_CHAR	signed char
MPI_SHORT	signed short int
MPI_INT	signed int
MPI_LONG	signed long int
MPI_UNSIGNED_CHAR	unsigned char
MPI_UNSIGNED_SHORT	unsigned short int
MPI_UNSIGNED	unsigned int
MPI_UNSIGNED_LONG	unsigned long int
MPI_FLOAT	float
MPI_DOUBLE	double
MPI_LONG_DOUBLE	long double
MPI_BYTE	
MPI_PACKED	

Типы MPI_BYTE и MPI_PACKED не имеют соответствия в языках C/C++.

Значением типа MPI_BYTE является байт. Байт не интерпретируется и отличен от символа.

Блокирующий прием сообщения

- Реализуется функцией

```
int MPI_Recv(void* buf, int count, MPI_Datatype datatype,  
int source, int tag, MPI_Comm comm, MPI_Status *status)
```

Здесь

`buf` – начальный адрес буфера процесса-получателя

`count` – число элементов в принимаемом сообщении

`datatype` – тип данных каждого элемента сообщения

`source` – номер процесса-отправителя

`tag` – тэг сообщения

`comm` – коммуникатор

`status` – параметры принятого сообщения

- Прием сообщения осуществляется, если его атрибуты соответствуют значениям источника, тэга и коммуникатора, которые указаны в операции приема. Процесс-получатель может задавать значение `MPI_ANY_SOURCE` для отправителя и/или значение `MPI_ANY_TAG` для тэга, определяя, что любой отправитель и/или тэг разрешен. Нельзя задать произвольное значение для коммуникатора `comm`. В случае успешного завершения возвращает предопределенное значение `MPI_SUCCESS`

- Функция `MPI_Recv` является блокирующей для процесса-получателя. Т.е. его выполнение приостанавливается до завершения работы этой функции.
- Вызов функции `MPI_Recv` не обязан быть согласован по времени с вызовом `MPI_Send`.
- Длина получаемого сообщения должна быть равна или меньше длины буфера получения, в противном случае будет иметь место ошибка переполнения. Если сообщение меньше размера буфера получения, то в нем модифицируются только ячейки, соответствующие длине сообщения.
- Допускается ситуация, когда имена источника и получателя совпадают, то есть процесс может посылать сообщение самому себе. Это небезопасно, поскольку может привести к блокировке процесса (deadlock)
- Источник или тэг принимаемого сообщения могут быть неизвестны, если в операции приема были использованы значения типа `*ANY*`.
- Тип аргумента `status` определяется MPI. Статусные переменные размещаются пользователем явно, то есть они не являются системными объектами.
- `MPI_Status` есть структура, которая содержит по крайней мере три поля, называемые `MPI_SOURCE`, `MPI_TAG` и `MPI_ERROR`. Следовательно, `status.MPI_SOURCE`, `status.MPI_TAG` и `status.MPI_ERROR` содержат источник, тэг и код ошибки принятого сообщения.

Коммуникационные режимы

- Вызов **send**, описанный в параграфе 3.2.1, является блокирующим: он не возвращает управления до тех пор, пока данные и атрибуты сообщения не будут сохранены в другом месте так, чтобы процесс-отправитель мог обращаться к буферу послыки и перезаписывать его.
- **Стандартный** коммуникационный режим используется в вызове **send**. В этом режиме решение о том, будет ли исходящее сообщение буферизовано или нет, принимает MPI. В случае буферизации случае операция послыки данных может завершиться до того, как будет вызван соответствующий прием. Если MPI отказывается от буферизации исходящего сообщения, вызов **send** не будет завершен, пока данные не будут перемещены в процесс-получатель.
- При **буферизованном** режиме операция послыки сообщения является локальной, и ее завершение не зависит от приема. Если послыка выполнена и никакого соответствующего приема не инициировано, то MPI буферизует исходящее сообщение, чтобы позволить завершиться вызову **send**. Если не имеется достаточного объема буферного пространства, будет иметь место ошибка. Объем буферного пространства задается пользователем.

- При **синхронном** режиме посылка может стартовать вне зависимости от того, был ли начат соответствующий прием. Посылка будет завершена успешно, только если соответствующая операция приема стартовала. Выполнение обмена в этом режиме не является локальным.
- При обмене **по готовности** посылка может быть запущена только тогда, когда прием уже инициирован. В противном случае операция является ошибочной, и результат будет неопределенным. Завершение операции посылки не зависит от состояния приема и указывает, что буфер посылки может быть повторно использован. Операция посылки, которая использует режим готовности, имеет ту же семантику, как и стандартная или синхронная передача. Это означает, что отправитель обеспечивает систему дополнительной информацией (именно, что прием уже инициирован).
- Для трех дополнительных коммуникационных режимов используются три дополнительные функции передачи. Коммуникационный режим отмечается одной префиксной буквой: **B** – для буферизованного, **S** – для синхронного и **R** – для режима по готовности.

```
int MPI_Bsend(void* buf, int count, MPI_Datatype datatype,  
              int dest, int tag, MPI_Comm comm);  
int MPI_Ssend(void* buf, int count, MPI_Datatype datatype,  
              int dest, int tag, MPI_Comm comm);  
int MPI_Rsend(void* buf, int count, MPI_Datatype datatype,  
              int dest, int tag, MPI_Comm comm);
```

Здесь

`buf` – начальный адрес буфера процесса-получателя

`count` – число элементов в принимаемом сообщении

`datatype` – тип данных каждого элемента сообщения

`dest` – номер процесса-получателя

`tag` – тэг сообщения

`comm` – коммуникатор

- Имеется только одна операция приема `MPI_Recv`, которая может соответствовать любому режиму передачи. Эта операция – блокирующая: она завершается только после того, когда приемный буфер уже содержит новое сообщение.

Семантика парного обмена

- **Очередность.** Сообщения не обгоняют друг друга: если отправитель последовательно посылает два сообщения в один адрес, то они принимаются в том же порядке. Если получатель инициирует два приема последовательно, то вторая операция не будет принимать сообщение, если первая операция все еще не выполнена. MPI-программа детерминирована, если процессы однопоточные и константа **MPI_ANY_SOURCE** не используется при приеме.
- **Продвижение обмена.** Если пара соответствующих посылки и приема инициализирована двумя процессами, то по крайней мере одна из этих двух операций будет завершена независимо от других действий в системе. Операция посылки сообщения гарантированно завершится, если только операция приема не завершится ранее за счет приема другого сообщения. Операция приема сообщения гарантированно завершится, если только данное сообщение не будет поглощено другой операцией приема в том же самом процессе.
- **Однозначность** выполнения коммуникаций в MPI обеспечивает программист.
- **Ограничение по ресурсам.** Любое выполнение операций обмена предполагает наличие ресурсов, которые могут быть ограниченными. Может иметь место ошибка, когда недостаток ресурсов ограничивает выполнение вызова.

Распределение и использование буферов

Пользователь может задать буфер, используемый для буферизации сообщений, посылаемых в режиме буферизации. Буферизация выполняется процессом-отправителем. Только один буфер может быть присоединен к процессу за один раз.

```
int MPI_Buffer_attach (void* buffer, int size);
```

Здесь

`buffer` – начальный адрес буфера;

`size` – размер буфера в байтах.

Адрес и размер отключенного буфера возвращаются операцией

```
int MPI_Buffer_detach(void* buffer_addr, int * size)
```

Операция будет блокирована, пока находящееся в буфере сообщение не будет передано.

Неблокирующий обмен

- Неплокирующий вызов инициирует операцию посылки, но не завершает ее. Управление возвращается в процесс перед тем, как сообщение будет послано из буфера отправителя. Проверка завершения посылки, т.е. извлечения данных из буфера отправителя, выполняется при помощи отдельного вызова.
- Во всех случаях вызов начала посылки является локальным: он заканчивается немедленно, безотносительно к состоянию других процессов.
- Аналогично, неплокирующий вызов инициирует операцию приема, но не завершает ее. Вызов будет закончен до записи сообщения в приемный буфер. Проверка завершения посылки, т.е. помещения данных в буфер получателя, выполняется при помощи отдельного вызова.
- Пересылка данных из памяти отправителя может выполняться параллельно с вычислениями, выполняемыми на процессе-отправителе после того, как передача была инициирована до ее завершения. Пересылка данных в память получателя может выполняться параллельно с вычислениями, производимыми в процессе-получателе после того, как прием был инициирован, и до его завершения.
- Использование неплокируемого приема позволяет избежать системной буферизации и дополнительного копирования информации.

- Неблокируемые вызовы начала посылки могут использовать четыре режима: стандартный, буферизуемый, синхронный и по готовности с сохранением их семантики.
- Передачи во всех режимах, исключая режим по готовности, могут стартовать вне зависимости от того, был ли инициирован соответствующий прием; неблокируемая посылка по готовности может быть начата, только если инициирован соответствующий прием.
- Неблокирующие передачи могут соответствовать блокирующим приемам и наоборот.
- Неблокирующие обмены используют скрытые **запросы**, чтобы идентифицировать операции обмена и сопоставить операцию, которая инициирует обмен с операцией, которая заканчивает его. Запросы являются системными объектами, которые становятся доступными в процессе обработки.

Инициализация обмена

Для функций, инициализирующих неблокирующий обмен информацией, используется префикс I (ImmEDIATE). Дополнительно, для обозначения буферизованного, синхронного режимов передачи данных или передачи данных по готовности используются префиксы B, S, R.

```
int MPI_Isend (void* buf, int count, MPI_Datatype datatype,  
              int dest, int tag, MPI_Comm comm, MPI_Request *request);  
int MPI_Ibsend (void* buf, int count, MPI_Datatype datatype,  
               int dest, int tag, MPI_Comm comm, MPI_Request *request);  
int MPI_Issend (void* buf, int count, MPI_Datatype datatype,  
               int dest, int tag, MPI_Comm comm, MPI_Request *request);  
int MPI_Irsend (void* buf, int count, MPI_Datatype datatype,  
               int dest, int tag, MPI_Comm comm, MPI_Request *request);
```

buf – начальный адрес буфера процесса-отправителя

count – число элементов в принимаемом сообщении

datatype – тип данных каждого элемента сообщения

dest – номер процесса-получателя

tag – тэг сообщения

comm – коммуникатор

request – дескриптор запроса обмена

```
int MPI_Irecv (void* buf, int count, MPI_Datatype datatype,  
               int source, int tag, MPI_Comm comm,  
               MPI_Request *request);
```

Здесь

`buf` – начальный адрес буфера процесса-получателя

`count` – число элементов в принимаемом сообщении

`datatype` – тип данных каждого элемента сообщения

`source` – номер процесса-отправителя

`tag` – тэг сообщения

`comm` – коммуникатор

`request` – параметры принятого сообщения

Завершение операции обмена

- Для завершения операций неблокирующего обмена используются следующие функции

```
int MPI_Wait(MPI_Request *request, MPI_Status *status);  
int MPI_Test(MPI_Request *request, int *flag,  
             MPI_Status *status);
```

здесь

`request` – дескриптор запроса (коммуникационного объекта);

`status` – статусный объект.

- Обращение к `MPI_WAIT` заканчивается, когда завершена операция, указанная в запросе. Если коммуникационный объект, связанный с этим запросом, был создан вызовом неблокирующей отправки или приема, то он удаляется, и дескриптор запроса устанавливается в `MPI_REQUEST_NULL`. Вызов возвращает в `status` информацию о завершенной операции. Разрешается вызывать `MPI_WAIT` с нулевым или неактивным аргументом запроса. В этом случае операция заканчивается немедленно со статусом `empty`. `MPI_WAIT` является нелокальной операцией.
- Обращение к `MPI_TEST` возвращает `flag = true`, если операция, указанная в запросе, завершена. `MPI_TEST` является локальной операцией.

Семантика неблокирующих операций

- **Очередность.** Операции неблокирующих коммуникаций упорядочены согласно порядку исполнения вызовов, которые инициируют обмен. Требование отсутствия обгона расширено на неблокирующий обмен.
- **Продвижение обмена.** Вызов `MPI_WAIT`, который завершает прием, будет в конечном итоге заканчиваться, если соответствующая посылка была начата и не закрыта другим приемом. Если соответствующая посылка неблокирующая, тогда прием должен завершиться, даже если отправитель не выполняет никакого вызова, чтобы завершить передачу. Аналогично, обращение к `MPI_WAIT`, которое завершает посылку, будет заканчиваться, если соответствующий прием инициирован.

Множественные завершения

- Вызовы `MPI_WAITANY` или `MPI_TESTANY` можно использовать для ожидания завершения одной из нескольких операций. Вызовы `MPI_WAITALL` или `MPI_TESTALL` могут быть использованы для всех ждущих операций в списке. Вызовы `WAITSOME` или `MPI_TESTSOME` можно использовать для завершения всех разрешенных операций в списке.

```
int MPI_Waitany (int count, MPI_Request *array_of_requests,  
                int *index, MPI_Status *status);
```

`count` – длина списка дескрипторов;

`array_of_requests` – массив дескрипторов;

`index` – индекс дескриптора для завершенной операции;

`status` – статусный объект.

- Операция блокирует работу до тех пор, пока не завершится одна из операций из массива активных запросов. Если более чем одна операция задействована и может закончиться, выполняется произвольный выбор. Операция возвращает в `index` индекс этого запроса в массиве и возвращает в `status` статус завершаемого обмена. Если запрос был создан операцией неблокирующего обмена, то он удаляется, и дескриптор запроса устанавливается в `MPI_REQUEST_NULL`.

- Если список не содержит активных дескрипторов, тогда вызов заканчивается немедленно с `index=MPI_UNDEFINED` и со статусом `empty`.

```
int MPI_Testany (int count, MPI_Request *array_of_requests,  
                int *index, int *flag, MPI_Status *status);
```

`count` – длина списка дескрипторов;

`array_of_requests` – массив дескрипторов;

`index` – индекс дескриптора для завершенной операции;

`flag = true`, если одна из операций завершена;

`status` – статусный объект.

Тестирует завершение либо одной, либо никакой из операций, связанных с активными дескрипторами. В первом случае возвращается `flag=true`, индекс этого запроса в массиве `index` и статус этой операции в `status`; если запрос был создан вызовом неблокирующего обмена, то запрос удаляется, и дескриптор устанавливается в `MPI_REQUEST_NULL`. В последнем случае (не завершено никакой операции) возвращается `flag=false`, значение `MPI_UNDEFINED` в `index` и состояние аргумента `status` является неопределенным. Массив может содержать нуль или неактивные дескрипторы. Если массив не содержит активных дескрипторов, то вызов заканчивается немедленно с `flag=true`, `index = MPI_UNDEFINED` и `status=empty`.

```
int MPI_Waitall(int count, MPI_Request array_of_requests[],
               MPI_Status array_of_statuses[]);
```

`count` – длина списка дескрипторов;

`array_of_requests` – массив дескрипторов;

`status` – массив статусный объект.

Блокирует работу, пока все операции обмена, связанные с активными дескрипторами в списке, не завершатся, и возвращает статус всех операций. Оба массива имеют то же самое количество элементов. Элемент с номером `i` в `array_of_statuses` устанавливается в возвращаемый статус `i`-й операции. Запросы, созданные операцией неблокирующего обмена, удаляются, и соответствующие дескрипторы устанавливаются в `MPI_REQUEST_NULL`. Список может содержать нуль или неактивные дескрипторы. Вызов устанавливает статус каждого такого элемента в состояние `empty`.

```
int MPI_Testall(int count, MPI_Request array_of_requests[],
               int *flag, MPI_Status array_of_statuses[]);
```

Ведет себя подобно `MPI_Waitall` за исключением того, что заканчивается немедленно. Возвращает `flag=true`, если завершены все операции, связанные с активными дескрипторами запроса

```
int MPI_Waitsome (int incount, MPI_Request *array_of_requests,  
                  int *outcount, int *array_of_indices,  
                  MPI_Status *array_of_statuses);
```

Здесь:

`incount` – длина массива запросов;

`array_of_requests` – массив запросов;

`outcount` – число завершенных запросов;

`array_of_indices` – массив индексов операций, которые завершены;

`array_of_statuses` – массив статусных операций для завершенных операций.

Ожидает, пока, по крайней мере, одна операция, связанная с активным дескриптором в списке, не завершится. Возвращает в `outcount` число запросов из списка `array_of_indices`, которые завершены. Возвращает в первых `outcount` элементах массива `array_of_indices` индексы этих операций (индекс внутри `array_of_requests`). Возвращает в первых `outcount` элементах массива `array_of_status` статус этих завершенных операций. Если завершенный запрос был создан вызовом неблокирующего обмена, то он удаляется, и связанный дескриптор устанавливается в `MPI_REQUEST_NULL`. Если список не содержит активных дескрипторов, то вызов заканчивается немедленно со значением `outcount=MPI_UNDEFINED`.

```
int MPI_Testsome (int incount, MPI_Request *array_of_requests,  
                  int *outcount, int *array_of_indices,  
                  MPI_Status *array_of_statuses);
```

Здесь:

`incount` – длина массива запросов;

`array_of_requests` – массив запросов;

`outcount` – число завершенных запросов;

`array_of_indices` – массив индексов операций, которые завершены;

`array_of_statuses` – массив статусных операций для завершенных операций.

Функция `MPI_TESTSOME` ведет себя подобно `MPI_WAIT SOME` за исключением того, что заканчивается немедленно. Если ни одной операции не завершено, она возвращает `outcount=0`. Если не имеется активных дескрипторов в списке, она возвращает `outcount=MPI_UNDEFINED`.

Проба и отмена

- Операции `MPI_PROBE` и `MPI_Iprobe` позволяют проверить входные сообщения без реального их приема. Пользователь затем может решить, как ему принимать эти сообщения, основываясь на информации, возвращенной аргументом `status`.

```
int MPI_Iprobe(int source, int tag, MPI_Comm comm,  
               int *flag, MPI_Status *status);
```

`source` – номер процесса-отправителя или `MPI_ANY_SOURCE`;

`tag` – значение тэга или `MPI_ANY_TAG`;

`comm` – коммуникатор;

`flag` – признак результата;

`status` – статус.

Возвращает `flag=true`, если имеется сообщение, которое может быть получено и которое соответствует образцу, описанному аргументами `source`, `tag`, и `comm`. Вызов соответствует тому же сообщению, которое было бы получено с помощью вызова `MPI_RECV (... , source, tag, comm, status)`, выполненного в той же точке программы, и возвращает статус с теми же значениями, которые были бы возвращены `MPI_RECV()`. В противном случае вызов возвращает `flag=false` и оставляет статус неопределенным.

```
int MPI_Probe(int source, int tag, MPI_Comm comm,  
             MPI_Status *status)
```

`source` – номер процесса-отправителя или `MPI_ANY_SOURCE`;

`tag` – значение тэга или `MPI_ANY_TAG`;

`comm` – коммуникатор;

`flag` – признак результата;

`status` – статус.

- `MPI_PROBE` ведет себя подобно `MPI_Iprobe`, исключая то, что функция `MPI_PROBE` является блокирующей и заканчивается после того, как соответствующее сообщение было найдено.
- Если обращение к `MPI_PROBE` уже было запущено процессом и посылка, которая соответствует пробе, уже инициирована тем же процессом, то вызов `MPI_PROBE` будет завершен, если сообщение не получено другой конкурирующей операцией приема (которая выполняется другим потоком опробуемого процесса). Аналогично, если процесс ожидает выполнения `MPI_Iprobe` и соответствующее сообщение было запущено, то обращение к `MPI_Iprobe` возвратит `flag=true`, если сообщение не получено другой конкурирующей приемной операцией.

- Инициация неблокирующих операций получения или отправки связывает пользовательские ресурсы, и может оказаться необходимой отмена, чтобы освободить эти ресурсы.

```
int MPI_Cancel(MPI_Request *request);
```

Маркирует для отмены ждущие неблокирующие операции обмена (передача или прием). Вызов является локальным. Он заканчивается немедленно, возможно перед действительной отменой обмена. После маркировки необходимо завершить эту операцию, используя вызов `MPI_WAIT` или `MPI_TEST`

- Должно выполняться следующее условие: либо отмена имеет успех, либо имеет успех обмен, но не обе ситуации вместе.

```
int MPI_Test_cancelled(MPI_Status *status, int *flag);
```

Возвращает `flag=true`, если обмен, связанный со статусным объектом, был отменен успешно. В таком случае поля статуса (такие как `count` или `tag`) не определены. В противном случае возвращается `flag=false`.

Совмещенные прием и передача сообщений

Следующая операция комбинирует в одном обращении посылку сообщения одному получателю и прием сообщения от другого отправителя.

```
int MPI_Sendrecv(void *sendbuf, int sendcount,  
    MPI_Datatype sendtype, int dest, int sendtag,  
    void *recvbuf, int recvcount, MPI_Datatype recvtype,  
    int source, MPI_Datatype recvtag, MPI_Comm comm,  
    MPI_Status *status);
```

`sendbuf` – начальный адрес буфера отправителя

`sendcount` – число элементов в буфере отправителя

`sendtype` – тип элементов в буфере отправителя

`dest` – номер процесса-получателя

`sendtag` – тэг процесса-отправителя

`recvbuf` – начальный адрес приемного буфера

`recvcount` – число элементов в приемном буфере

`recvtype` – тип элементов в приемном буфере

`source` – номер процесса-отправителя

`recvtag` – тэг процесса-получателя

`comm` – коммуникатор; `status` – статус.

- Получателем и отправителем может быть тот же самый процесс.
- Семантика операции `send-receive` похожа на запуск двух конкурирующих потоков, один выполняет передачу, другой – прием, с последующим объединением этих потоков.
- Обмен с процессом, который имеет значение `MPI_PROC_NULL`, не приводит к выполнению каких-либо действий.
- Передача в процесс с `MPI_PROC_NULL` успешна и заканчивается сразу, как только возможно.
- Прием от процесса с `MPI_PROC_NULL` успешен и заканчивается сразу, как только возможно, без изменения буфера приема.

Отложенные (persistent) запросы на взаимодействие

- Часто требуется выполнять обмены с одинаковыми параметрами (например, в цикле). В этом случае можно один раз инициализировать операцию обмена и потом ее выполнять, не тратя на каждой итерации дополнительное время на инициализацию внутренних структур данных.
- При этом способ приема сообщения никак не зависит от способа его отправки: сообщение, отправленное с помощью отложенных запросов, может быть принято как обычным образом, так и с помощью отложенных запросов.
- Создание дескриптора запроса, который может быть использован для выполнения отложенных операций взаимодействия, реализуется функциями

```
int MPI_Send_init(const void* buf, int count, MPI_Datatype datatype, int dest,  
int tag, MPI_Comm comm, MPI_Request *request);
```

```
int MPI_Bsend_init(const void* buf, int count, MPI_Datatype datatype, int dest,  
int tag, MPI_Comm comm, MPI_Request *request);
```

```
int MPI_Ssend_init(const void* buf, int count, MPI_Datatype datatype, int dest,  
int tag, MPI_Comm comm, MPI_Request *request);
```

```
MPI_Rsend_init(const void* buf, int count, MPI_Datatype datatype, int dest,  
int tag, MPI_Comm comm, MPI_Request *request);
```

`int MPI_Recv_init(void* buf, int count, MPI_Datatype datatype, int source, int tag, MPI_Comm comm, MPI_Request *request)`

- При этом лишь создается дескриптор, но сам обмен не инициализируется.
- Инициализация отложенного запроса на взаимодействия реализуется функциями

`int MPI_Start(MPI_Request *request);`

- запуск операции, соотв. одному дескриптору запроса

`int MPI_Startall(int count, MPI_Request array_of_requests[])`

- запуск нескольких операций одновременно для массива дескрипторов

- Завершение отложенных операций взаимодействия проверяется операциями `MPI_WAIT`, `MPI_TEST`, `MPI_WAITANY`, `MPI_TESTANY`, `MPI_WAITALL`, `MPI_TESTALL`, `MPI_WAIT SOME`, `MPI_TEST SOME`

- При этом, в отличие от неблокирующих операций обмена, по завершении операции, запущенной при помощи отложенного запроса на взаимодействие, дескриптор запроса не аннулируется и может быть использован повторно.

Производные типы данных

- Под производным типом данных в MPI понимают описание набора значений предусмотренного в MPI типа, причем в общем случае описываемые значения необязательно непрерывно располагаются в памяти.
- Задание типа в MPI принято осуществлять при помощи т.н. карты типа (type map) в виде последовательности входящих в тип значений; каждое отдельное значение описывается указанием типа и смещением адреса месторасположения относительно некоторого базового адреса

$$\text{TypeMap} = \{(\text{type}_0, \text{disp}_0), (\text{type}_1, \text{disp}_1), \dots, (\text{type}_{n-1}, \text{disp}_{n-1})\}$$

- Часть карты типа с указанием только типов значений именуется в MPI сигнатурой типа.

$$\text{TypeSignature} = \{\text{type}_0, \text{type}_1, \dots, \text{type}_{n-1}\}$$

Входящие в сообщение данные

double a; //адрес 32

double b; //адрес 48

int n; //адрес 64

Карта типа

{(MPI_DOUBLE, 0),

(MPI_DOUBLE, 16),

(MPI_INT, 32)}

- Не требуется, чтобы смещения были положительными, различными или расположенными по возрастанию. Порядок объектов не обязан совпадать с их порядком в памяти, и объект может появляться более чем один раз.

Нижняя граница типа представляет собой смещение его первого байта.

Верхняя граница типа представляет собой смещение для байта, располагающегося вслед за последним байтом типа, с поправкой на выравнивание адресов данных.

Протяженность (экстент) – это размер памяти в байтах, которую необходимо отвести под хранение производного типа данных, с поправкой на выравнивание адресов данных.

Размер типа данных – это число байт, которые занимают данные (разность между адресами последнего и первого байта данных).

Функция

```
int MPI_Address(void* location, MPI_Aint *address);
```

в переменной `address` возвращает числовой эквивалент адреса `location`

Функция

```
int MPI_Type_extent(MPI_Datatype datatype, MPI_Aint *extent);
```

в переменной `extent` возвращает протяженность, или экстент, типа данных `datatype`.

Функция

```
int MPI_Type_size(MPI_Datatype datatype, int *size);
```

в переменной `size` возвращает размер типа данных `datatype`.

Функция

`int MPI_Type_lb(MPI_Datatype datatype, MPI_Aint* displacement);`
возвращает в переменной `displacement` нижнюю границу типа `datatype`.

Функция

`int MPI_Type_ub(MPI_Datatype datatype, MPI_Aint* displacement);`
возвращает в переменной `displacement` верхнюю границу типа `datatype`.

Конструирование производных типов данных

- Непрерывный способ позволяет определить непрерывный набор элементов существующего исходного типа как новый тип

`int MPI_Type_contiguous(int count, MPI_Datatype oldtype,
MPI_Datatype *newtype);`

`count` – число повторений (неотрицательное целое)

`oldtype` – старый тип данных (дескриптор)

`newtype` – новый тип данных (дескриптор)

Новый тип `newtype` есть тип, полученный конкатенацией (сцеплением) `count` копий старого типа `oldtype`.

- Векторный способ обеспечивает создание нового производного типа как набора элементов существующего типа, между которыми имеются регулярные промежутки по памяти.

```
int MPI_Type_vector(int count, int blocklength,  
    int stride, MPI_Datatype oldtype, MPI_Datatype *newtype);
```

count – число блоков (неотрицательное целое)

blocklength – число элементов в каждом блоке (неотрицательное целое)

stride – число элементов между началом каждого блока (целое)

oldtype – исходный тип данных (дескриптор)

newtype – новый тип данных (дескриптор)

```
int MPI_Type_hvector(int count, int blocklength,  
    MPI_Aint stride, MPI_Datatype oldtype, MPI_Datatype *newtype);
```

count – число блоков (неотрицательное целое)

blocklength – число элементов в каждом блоке (неотрицательное целое)

stride – число байт между началом каждого блока (целое)

oldtype – исходный тип данных (дескриптор)

newtype – новый тип данных (дескриптор)

- Индексный способ. Следующая функция позволяет реплицировать исходный тип в последовательность блоков (каждый блок есть конкатенация исходного типа), где каждый блок может содержать различное число копий и иметь различное смещение. Все смещения блоков кратны длине исходного типа.

```
int MPI_Type_indexed(int count, int array_of_blocklengths[],  
    int array_of_displacements[], MPI_Datatype oldtype,  
    MPI_Datatype *newtype);
```

где

`count` – число блоков;

`array_of_blocklengths` – число элементов в каждом блоке (массив неотрицательных целых);

`array_of_displacements` – смещение для каждого блока (массив целых);

`oldtype` – исходный тип данных (дескриптор);

`newtype` – новый тип данных (дескриптор).

Функция

```
int MPI_Type_hindexed(int count, int *array_of_blocklengths,  
    MPI_Aint *array_of_displacements, MPI_Datatype oldtype,  
    MPI_Datatype *newtype);
```

аналогична ранее рассмотренной функции `MPI_Type_indexed`, с той разницей, что смещение для каждого блока задается в байтах.

- Структурный способ обеспечивает самое общее описание структурного типа через явное указание карты создаваемого типа.

```
int MPI_Type_struct(int count, int *array_of_blocklengths,  
    MPI_Aint *array_of_displacements,  
    MPI_Datatype *array_of_types, MPI_Datatype *newtype);
```

Здесь

`count` – число блоков;

`array_of_blocklength` – число элементов в каждом блоке;

`array_of_displacements` – смещение каждого блока в байтах;

`array_of_types` – тип элементов в каждом блоке (массив дескрипторов объектов типов данных)

`newtype` – новый тип данных (дескриптор)

`MPI_Type_struct` является общим типом конструктора. Он отличается от предыдущего тем, что позволяет каждому блоку состоять из репликаций различного типа.

Объявление и удаление производных типов данных

- Перед использованием созданный тип `datatype` должен быть объявлен, или зарегистрирован, при помощи функции

```
int MPI_Type_commit(MPI_Datatype *datatype);
```

При завершении использования производный тип `datatype` должен быть аннулирован при помощи функции

```
int MPI_Type_free(MPI_Datatype *datatype);
```

Упаковка и распаковка данных

В MPI предусмотрены способы упаковки и распаковки данных до/после передачи сообщений. Упаковываемые/распаковываемые данные могут характеризоваться разными типами и могут располагаться в разных адресах памяти. Упаковка данных реализуется функцией

```
int MPI_Pack(void* inbuf, int incount, MPI_Datatype datatype,  
            void *outbuf, int outsize, int *position, MPI_Comm comm);
```

`inbuf` – начало входного буфера;

`incount` – число единиц входных данных;

`datatype` – тип данных каждой входной единицы;

`outbuf` – начало выходного буфера;

`outsize` – размер выходного буфера в байтах;

`position` – текущая позиция в выходном буфере в байтах;

`comm` – коммуникатор для упакованного сообщения.

Входное значение **position** есть первая ячейка в выходном буфере, которая должна быть использована для упаковки. **position** увеличивается на размер упакованного сообщения, выходное значение **position** есть первая ячейка в выходном буфере, следующая за ячейками, занятыми упакованным сообщением. Аргумент **comm** – коммуникатор, используемый для передачи упакованного сообщения.

Упакованный объект может быть послан типом `MPI_PACKED`. Любая парная или коллективная коммуникационная функция может быть использована для передачи последовательности байтов, которая формирует упакованный объект, из одного процесса в другой. Этот упакованный объект также может быть получен любой приемной операцией, поскольку типовые правила соответствия ослаблены для сообщений, посланных с помощью `MPI_PACKED`. Сообщение, посланное с любым типом может быть получено с помощью `MPI_PACKED`. Такое сообщение может быть распаковано обращением к следующей функции

```
int MPI_Unpack(void* inbuf, int insize, int *position,  
              void *outbuf, int outcount,  
              MPI_Datatype datatype, MPI_Comm comm);
```

Здесь

`inbuf` – начало входного буфера;

`insize` – размер входного буфера в байтах;

`position` – текущая позиция в байтах;

`outbuf` – начало выходного буфера;

`outcount` – число единиц для распаковки;

`datatype` – тип данных каждой выходной единицы данных;

`comm` – коммуникатор для упакованных сообщений.

- Функция `MPI_UNPACK` распаковывает сообщение в приемный буфер, описанный аргументами `outbuf`, `outcount`, `datatype` из буферного пространства, описанного аргументами `inbuf` и `insize`. Входной буфер есть смежная область памяти, содержащая `insize` байтов, начиная с адреса `inbuf`. Входное значение `position` есть первая ячейка во входном буфере, занятом упакованным сообщением. `position` инкрементируется размером упакованного сообщения, так что выходное значение `position` есть первая ячейка во входном буфере после ячеек, занятых сообщением, которое было упаковано. `comm` есть коммуникатор для приема упакованного сообщения.

Для определения необходимого для упаковки данных размера буфера используется функция

```
int MPI_Pack_size(int incount, MPI_Datatype datatype,  
                  MPI_Comm comm, int *size);
```

Здесь

`incount` – количество элементов в буфере;

`datatype` – тип данных для упаковываемых элементов;

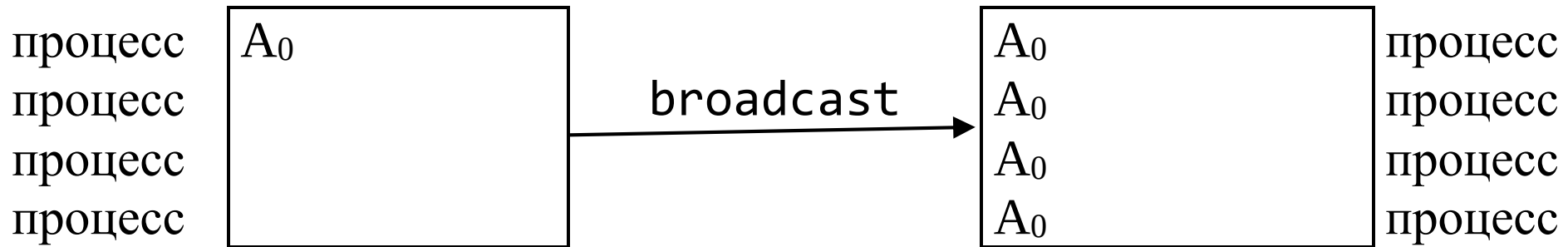
`comm` – коммуникатор для упакованного сообщения;

`size` – рассчитанный размер буфера.

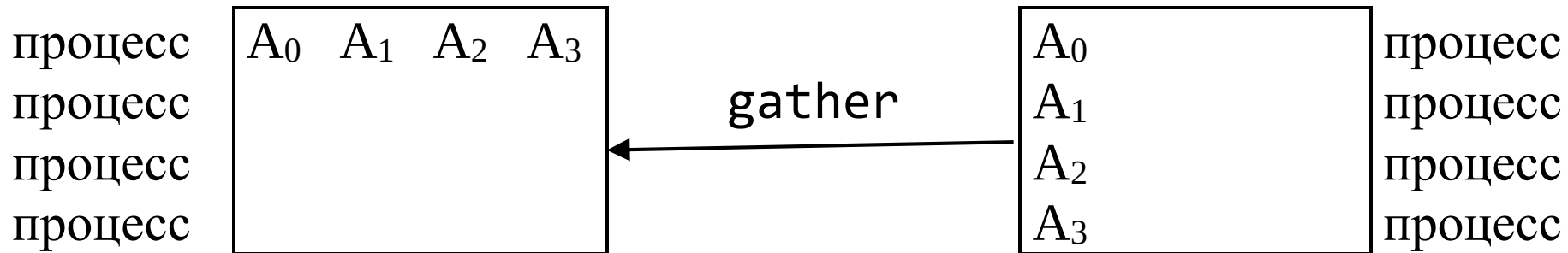
КОЛЛЕКТИВНЫЕ ВЗАИМОДЕЙСТВИЯ ПРОЦЕССОВ

К операциям коллективного обмена относятся

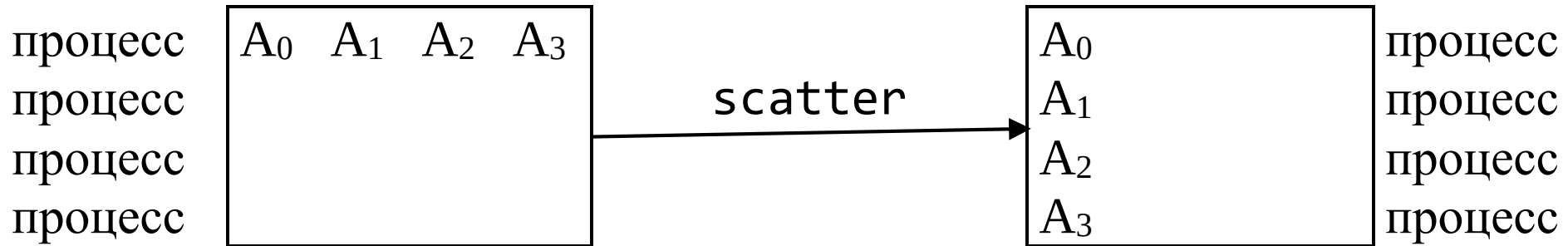
- Барьерная синхронизация всех процессов группы;
- Широковещательная передача (broadcast) от одного процесса всем остальным процессам группы;



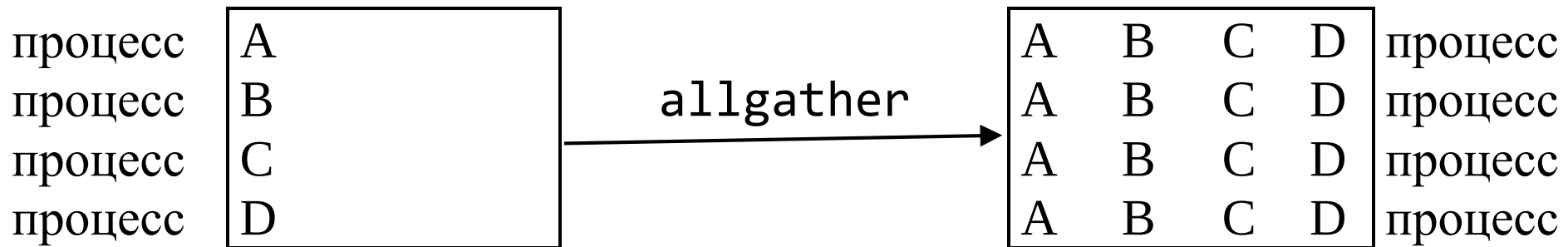
- Сбор данных из всех процессов группы в один процесс



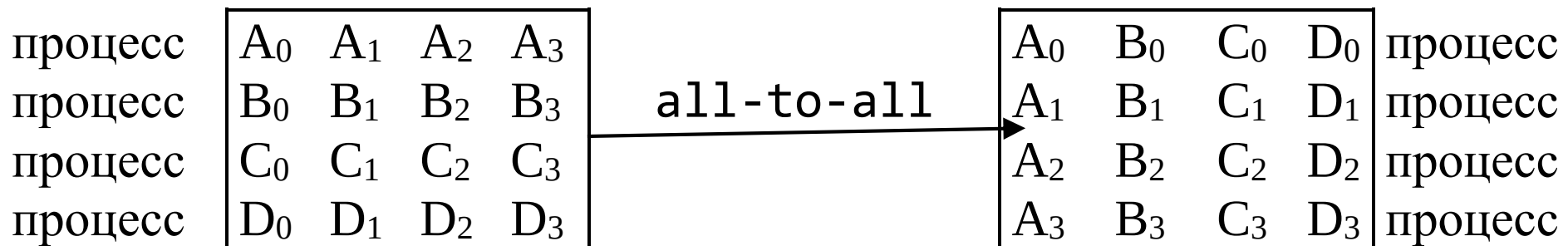
- Рассылка данных одним процессом группы всем процессам группы.



- Сбор данных, когда все процессы группы получают результат (allgather).



- Раздача/сбор данных из всех процессов группы всем процессам группы. Этот вариант называется также полным обменом (all-to-all).



- Глобальные операции редукции, такие как сложение, нахождение максимума, минимума или определенные пользователем функции, где результат возвращается всем процессам группы или в один процесс;
- Составная операция редукции и раздачи;
- Префиксная операция редукции, при которой в процессе j появляется результат редукции $0, 1, \dots, j, j \leq n$, где n – число процессов в группе.
- Коллективная операция выполняется путем вызова всеми процессами в группе коммуникационных функций с соответствующими аргументами. В коллективных операциях используются основные типы данных, и они должны совпадать у процесса-отправителя и процесса-получателя.
- Один из ключевых аргументов – это коммуникатор, который определяет группу участвующих в обмене процессов и обеспечивает контекст для этой операции.
- Различные коллективные операции (широковещание, сбор данных) имеют единственный процесс-отправитель или процесс-получатель. Такие процессы называются корневыми (root). Некоторые аргументы в коллективных функциях определены как “существенные только для корневого процесса” и игнорируются для всех других участников операции.

- Для коллективных операций количество посланных данных должно точно соответствовать количеству данных, описанных в процессе-получателе.
- Блокирующая операция завершается при выходе из соответствующей функции. Неблокирующие операции требуют отдельного вызова для своего завершения.
- Вызовы коллективных операций могут использовать те же коммуникаторы, что и парный обмен, при этом MPI гарантирует, что сообщения, созданные коллективными операциями, не будут смешаны с сообщениями, созданными парным обменом.
- Ключевым понятием в коллективных функциях является группа участвующих процессов, но в качестве явного аргумента выступает коммуникатор.
- Коммуникатор понимается как идентификатор группы, связанный с контекстом. Не разрешается использовать в качестве аргумента коллективной функции интер-коммуникатор (коммуникатор, соединяющий две группы).
- Функция барьерной синхронизации

`int MPI_Barrier(MPI_Comm comm);`

блокирует вызывающий процесс, пока все процессы группы не вызовут ее. В каждом процессе управление возвращается только тогда, когда все процессы в группе вызовут процедуру.

Широковещательный обмен

Функция широковещательной передачи посылает сообщение из корневого процесса всем процессам группы, включая себя.

```
int MPI_Bcast(void* buffer, int count, MPI_Datatype datatype,  
              int root, MPI_Comm comm );
```

`buffer` – адрес начала буфера

`count` – количество записей в буфере

`datatype` – тип данных в буфере

`root` – номер корневого процесса

`comm` – коммуникатор

- Функция вызывается всеми процессами группы с одинаковыми аргументами для `comm`, `root`. В момент возврата управления содержимое корневого буфера обмена будет уже скопировано во все процессы.
- В аргументе `datatype` можно задавать производные типы данных.
- Сигнатура типа данных `count`, `datatype` любого процесса обязана совпадать с соответствующей сигнатурой в корневом процессе. Это требует, чтобы количество посланных и полученных данных совпадало попарно для корневого и каждого другого процессов. Такое ограничение имеют и все другие коллективные операции, выполняющие перемещение данных.

Сбор данных

При выполнении операции сбора данных каждый процесс, включая корневой, посылает содержимое своего буфера в корневой процесс.

```
int MPI_Gather(void* sendbuf, int sendcount,  
              MPI_Datatype sendtype, void* recvbuf, int recvcount,  
              MPI_Datatype recvtype, int root, MPI_Comm comm);
```

`sendbuf` – начальный адрес буфера процесса-отправителя;

`sendcount` – количество элементов в отсылаемом сообщении;

`sendtype` – тип элементов в отсылаемом сообщении;

`recvbuf` – начальный адрес буфера процесса сборки данных (существенен только для корневого процесса)

`recvcount` – количество элементов в принимаемом сообщении (имеет значение только для корневого процесса)

`recvtype` – тип данных элементов в буфере процесса-получателя (дескриптор)

`root` – номер процесса-получателя

`comm` – коммуникатор (дескриптор)

- Корневой процесс получает сообщения, располагая их в порядке возрастания номеров процессов.

- Аргумент `recvcount` в главном процессе показывает количество элементов, которые он получил от каждого процесса, а не общее количество полученных элементов.
- В общем случае как для `sendtype`, так и для `recvtype` разрешены производные типы данных. Сигнатура типа данных `sendcount`, `sendtype` у процесса `j` должна быть такой же, как сигнатура `recvcount`, `recvtype` корневого процесса. Это требует, чтобы количество посланных и полученных данных совпадало попарно для корневого и каждого другого процессов. Разрешается различие в картах типов между отправителями и получателями.
- Описанные в функции `MPI_GATHER` количества и типы данных не должны являться причиной того, чтобы любая ячейка корневого процесса записывалась бы более одного раза.

Функция `MPI_GATHERV` позволяет принимать от каждого процесса переменное число элементов данных. Поэтому в функции входной параметр `recvcounts` является массивом. Она также обеспечивает большую гибкость в размещении данных в корневом процессе. Для этой цели используется новый входной аргумент `displs`, определяющий смещения данных относительно начала принимающего буфера.

```
int MPI_Gatherv(const void* sendbuf, int sendcount,  
               MPI_Datatype sendtype, void* recvbuf,  
               const int recvcounts[], const int displs[],  
               MPI_Datatype recvttype, int root, MPI_Comm comm);
```

Здесь:

`sendbuf` – начальный адрес буфера процесса-отправителя

`sendcount` – количество элементов в отсылаемом сообщении

`sendtype` – тип элементов в отсылаемом сообщении

`recvbuf` – начальный адрес буфера процесса сборки данных (существенно для корневого процесса)

`recvcounts` – массив целых чисел (по размеру группы), содержащий количество элементов, которые получены от каждого из процессов (используется корневым процессом)

`displs` – массив целых чисел (по размеру группы). Элемент `j` определяет смещение относительно `recvbuf`, в котором размещаются данные из процесса `j` (используется корневым процессом)

`recvttype` – тип данных элементов в буфере процесса-получателя

`root` – номер процесса-получателя

`comm` – коммуникатор

- Сообщения помещаются в принимающий буфер корневого процесса в порядке возрастания их номеров, то есть данные, посланные процессом j , помещаются в j -ю часть принимающего буфера `recvbuf` на корневом процессе. j -я часть `recvbuf` начинается со смещения `displs[j]`.
- Номер принимающего буфера игнорируется во всех некорневых процессах.
- Сигнатура типа, используемая `sendcount`, `sendtype` в процессе j должна быть такой же, как и сигнатура, используемая `recvcounts[i]`, `recvtype` в корневом процессе. Это требует, чтобы количество посланных и полученных данных совпадало попарно для корневого и каждого другого процессов. Разрешается различие в картах типов между отправителями и получателями.
- В корневом процессе используются все аргументы функции `MPI_GATHERV`, а на всех других процессах используются только аргументы `sendbuf`, `sendcount`, `sendtype`, `root`, `comm`. Переменные `comm` и `root` должны иметь одинаковые значения во всех процессах.
- Описанные в функции `MPI_GATHERV` количества, типы данных и смещения не должны приводить к тому, чтобы любая область корневого процесса записывалась бы более одного раза.

Рассылка

Операция `MPI_Scatter` обратна к операции `MPI_Gather`.

```
int MPI_Scatter (void* sendbuf, int sendcount,  
                MPI_Datatype sendtype, void* recvbuf, int recvcount,  
                MPI_Datatype recvtype, int root, MPI_Comm comm);
```

Здесь:

`sendbuf` – начальный адрес буфера рассылки (используется только корневым процессом)

`sendcount` – количество элементов, посылаемых каждому процессу (используется только корневым процессом)

`sendtype` – тип данных элементов в буфере посылки (используется только корневым процессом)

`recvbuf` – адрес буфера процесса-получателя

`recvcount` – количество элементов в буфере корневого процесса

`recvtype` – тип данных элементов приемного буфера

`root` – номер процесса-получателя

`comm` – коммуникатор

- Буфер отправки игнорируется всеми некорневыми процессами.
- Корневой процесс использует все аргументы функции, а другие процессы используют только аргументы `recvbuf`, `recvcount`, `recvtype`, `root`, `comm`.
- Аргументы `root` и `comm` должны быть одинаковыми во всех процессах.
- Сигнатура типа, связанная с `sendcount`, `sendtype` в корневом процессе должна совпадать с сигнатурой типа, связанной с `recvcount`, `recvtype` во всех процессах. Это требует, чтобы количество посланных и полученных данных совпадало попарно для корневого и каждого другого процессов. Разрешается различие в картах типов между отправителями и получателями.
- Описанные в функции `MPI_SCATTER` количества и типы данных не должны являться причиной того, чтобы любая ячейка корневого процесса записывалась бы более одного раза.

Операция MPI_SCATTERV обратна операции MPI_GATHERV

```
int MPI_Scatterv(const void* sendbuf, const int sendcounts[],  
               const int displs[], MPI_Datatype sendtype, void* recvbuf,  
               int recvcount, MPI_Datatype recvtype, int root,  
               MPI_Comm comm);
```

Здесь:

sendbuf – адрес буфера отправки (используется только корневым процессом)

sendcounts – целочисленный массив (размера группы), определяющий число элементов, для отправки каждому процессу

displs – целочисленный массив (размера группы). Элемент *j* указывает смещение (относительно **sendbuf**, из которого берутся данные для процесса)

sendtype – тип элементов посылающего буфера

recvbuf – адрес принимающего буфера

recvcount – число элементов в посылающем буфере

recvtype – тип данных элементов принимающего буфера

root – номер посылающего процесса

comm – коммуникатор (дескриптор)

- Аргумент `sendbuf` игнорируется во всех некорневых процессах.
- Сигнатура типа, связанная с `sendcount[j]`, `sendtype` в главном процессе, должна быть той же, что и сигнатура, связанная с `recvcount`, `recvtype` в процессе `j`. Это требует, чтобы количество посланных и полученных данных совпадало попарно для корневого и каждого другого процессов. Разрешается различие в картах типов между отправителями и получателями.
- Корневой процесс использует все аргументы функции, а другие процессы используют только аргументы `recvbuf`, `recvcount`, `recvtype`, `root`, `comm`.
- Аргументы `root` и `comm` должны быть одинаковыми во всех процессах.
- Описанные в функции `MPI_SCATTER` количества и типы данных не должны приводить к тому, чтобы любая область памяти корневого процесса записывалась бы более одного раза.

Сбор для всех процессов

```
int MPI_Allgather(const void* sendbuf, int sendcount,  
    MPI_Datatype sendtype, void* recvbuf, int recvcount,  
    MPI_Datatype recvtype, MPI_Comm comm);
```

`sendbuf` – начальный адрес посылающего буфера

`sendcount` – количество элементов в буфере

`sendtype` – тип данных элементов в посылающем буфере

`recvbuf` – адрес принимающего буфера

`recvcount` – количество элементов, полученных от любого процесса

`recvtype` – тип данных элементов принимающего буфера

`comm` – коммуникатор

Функцию `MPI_ALLGATHER` можно представить как функцию `MPI_GATHER`, где результат принимают все процессы, а не только главный. Блок данных, посланный j -м процессом, принимается каждым процессом и помещается в j -й блок буфера `recvbuf`. Правила использования `MPI_ALLGATHER` соответствуют правилам для `MPI_GATHER`. Сигнатура типа, связанная с `sendcount`, `sendtype` должна совпадать с сигнатурой типа, связанной с `recvcount`, `recvtype` во всех процессах.

Операцию `MPI_Allgatherv` можно представить как `MPI_Gatherv`, но при ее использовании результат получают все процессы, а не только один корневой.

```
int MPI_Allgatherv(const void* sendbuf, int sendcount,  
MPI_Datatype sendtype, void* recvbuf, const int recvcounts[],  
const int displs[], MPI_Datatype recvtype, MPI_Comm comm);
```

`sendbuf` – начальный адрес посылающего буфера

`sendcount` – количество элементов в посылающем буфере (целое)

`sendtype` – тип данных элементов в посылающем буфере (дескриптор)

`recvbuf` – адрес принимающего буфера (альтернатива)

`recvcounts` – целочисленный массив (размера группы), содержащий количество элементов, полученных от каждого процесса

`displs` – целочисленный массив (размера группы). Элемент `j` представляет смещение области (относительно `recvbuf`), где помещаются принимаемые данные от процесса `j`

`recvtype` – тип данных элементов принимающего буфера

`comm` – коммуникатор

`j`-й блок данных, посланный каждым процессом, принимается каждым процессом и помещается в `j`-й блок буфера `recvbuf`. Эти блоки не обязаны быть одинакового размера.

Операция all-to-all Scatter/Gather

`MPI_Alltoall` – расширение функции `MPI_Allgather` для случая, когда каждый процесс посылает различные данные каждому получателю.

```
int MPI_Alltoall(const void* sendbuf, int sendcount,  
                MPI_Datatype sendtype, void* recvbuf, int recvcount,  
                MPI_Datatype recvtype, MPI_Comm comm);
```

`sendbuf` – начальный адрес посылающего буфера

`sendcount` – количество элементов посылаемых в каждый процесс

`sendtype` – тип данных элементов посылающего буфера

`recvbuf` – адрес принимающего буфера

`recvcount` – количество элементов, принятых от какого-либо процесса

`recvtype` – тип данных элементов принимающего буфера

`comm` – коммуникатор

j-й блок, посланный процессом *i*, принимается процессом *j* и помещается в *i*-й блок буфера `recvbuf`.

Во всех процессах сигнатура типа, связанная с `sendcount`, `sendtype` должна быть той же, что и сигнатура, связанная с `recvcount`, `recvtype`. Количество посланных и полученных данных должно совпадать. Разрешается различие в картах типов между отправителями и получателями.

Следующая операция аналогична рассмотренной выше, однако пересылаемые блоки не обязаны быть одного размера.

```
int MPI_Alltoallv(const void* sendbuf, const int sendcounts[],  
const int sdispls[], MPI_Datatype sendtype, void* recvbuf,  
const int recvcounts[], const int rdispls[],  
MPI_Datatype recvtype, MPI_Comm comm);
```

`sendbuf` – начальный адрес посылающего буфера

`sendcounts` – целочисленный массив (размера группы), определяющий количество посылаемых каждому процессу элементов

`sdispls` – целочисленный массив (размера группы). Элемент j содержит смещение области (относит. `sendbuf`), из которой берутся данные для процесса j

`sendtype` – тип данных элементов посылающего буфера (дескриптор)

`recvbuf` – адрес принимающего буфера

`recvcounts` – целочисленный массив (размера группы), содержит число элементов, которые могут быть приняты от каждого процесса

`rdispls` – целочисленный массив (размера группы). Элемент i определяет смещение области (относительно `recvbuf`), в которой размещаются данные, получаемые из процесса i

`recvtype` – тип данных элементов принимающего буфера

`comm` – коммуникатор

Глобальные операции редукции

Глобальные операции редукции: суммирование, нахождение максимума, логическое «И», и т.д. – выполняются для всех процессов в группе. Операция редукции может быть одной из предопределенного списка операций или определяться пользователем. Функции глобальной редукции имеют несколько разновидностей: операции, возвращающие результат в один узел; функции, возвращающие результат во все узлы; операции просмотра.

Следующая функция объединяет элементы входного буфера каждого процесса в группе, используя операцию `op`, и возвращает объединенные значения в выходном буфере процесса с номером `root`.

```
int MPI_Reduce(void* sendbuf, void* recvbuf, int count,  
               MPI_Datatype datatype, MPI_Op op, int root, MPI_Comm comm);
```

`sendbuf` – адрес посылающего буфера

`recvbuf` – адрес принимающего буфера (используется только корневым процессом)

`count` – количество элементов в посылающем буфере

`datatype` – тип данных элементов посылающего буфера

`root` – номер главного процесса

`comm` – коммуникатор

- Буфер ввода определен аргументами `sendbuf`, `count` и `datatype`; буфер вывода определен параметрами `recvbuf`, `count` и `datatype`; оба буфера имеют одинаковое число элементов одинакового типа.
- Функция вызывается всеми членами группы с одинаковыми аргументами `count`, `datatype`, `op`, `root` и `comm`. Таким образом, все процессы имеют входные и выходные буфера одинаковой длины и с элементами одного типа.
- Каждый процесс может содержать один элемент или последовательность элементов; в последнем случае операция выполняется над всеми элементами в этой последовательности.

Предопределенные операции редукции

Имя	Значение
MPI_MAX	максимум
MPI_MIN	минимум
MPI_SUM	сумма
MPI_PROD	произведение
MPI LAND	логическое И
MPI_BAND	поразрядное И
MPI_LOR	логическое ИЛИ
MPI_BOR	поразрядное ИЛИ
MPI_LXOR	логическое исключающее ИЛИ
MPI_BXOR	поразрядное исключающее ИЛИ
MPI_MAXLOC	максимальное значение и местонахождения
MPI_MINLOC	минимальное значение и местонахождения

Типы данных для предопределенных операций редукции

Операция	Тип
MPI_MAX, MPI_MIN	MPI_INT, MPI_LONG, MPI_SHORT, MPI_UNSIGNED_SHORT, MPI_UNSIGNED, MPI_UNSIGNED_LONG, MPI_FLOAT, MPI_DOUBLE, MPI_REAL, MPI_LONG_DOUBLE
MPI_SUM, MPI_PROD	MPI_INT, MPI_LONG, MPI_SHORT, MPI_UNSIGNED_SHORT, MPI_UNSIGNED, MPI_UNSIGNED_LONG, MPI_FLOAT, MPI_DOUBLE, MPI_REAL, MPI_LONG_DOUBLE, MPI_COMPLEX
MPI_LAND, MPI_LOR, MPI_LXOR	MPI_INT, MPI_LONG, MPI_SHORT, MPI_UNSIGNED_SHORT, MPI_UNSIGNED, MPI_UNSIGNED_LONG
MPI_BAND, MPI_BOR, MPI_BXOR	MPI_INT, MPI_LONG, MPI_SHORT, MPI_UNSIGNED_SHORT, MPI_UNSIGNED, MPI_UNSIGNED_LONG, MPI_BYTE

Операции MINLOC и MAXLOC

Оператор `MPI_MINLOC` используется для расчета глобального минимума и соответствующего ему (наименьшего) номера процесса, в котором содержится данное значение. `MPI_MAXLOC` аналогично считает глобальный максимум и (наименьший) номер процесса, в котором содержится данное значение. Обе операции ассоциативны и коммутативны. Чтобы использовать `MPI_MINLOC` и `MPI_MAXLOC` в операции редукции, нужно обеспечить аргумент `datatype`, который представляет пару (значение и индекс). MPI предоставляет для этой цели ряд predefined типов.

Имя	Описание
<code>MPI_FLOAT_INT</code>	переменные типа <code>float</code> и <code>int</code>
<code>MPI_DOUBLE_INT</code>	переменные типа <code>double</code> и <code>int</code>
<code>MPI_LONG_INT</code>	переменные типа <code>long</code> и <code>int</code>
<code>MPI_2INT</code>	пара переменных типа <code>int</code>
<code>MPI_SHORT_INT</code>	переменные типа <code>short</code> и <code>int</code>
<code>MPI_LONG_DOUBLE_INT</code>	переменные типа <code>long double</code> и <code>int</code>

Задание пользовательской операции редукции

Для сопоставления соответствующему дескриптору некоторой пользовательской функции, реализующей операцию редукции, используется следующий вызов

```
int MPI_Op_create(MPI_User_function* user_fn, int commute,  
                  MPI_Op* op);
```

Здесь:

`user_fn` – пользовательская функция;

`commute` – `true`, если операция, задаваемая пользовательской функцией, является коммутативной; `false` в противном случае;

`op` – дескриптор операции.

Подразумевается, что в любом случае операция, задаваемая пользовательской функцией, является ассоциативной.

Прототип пользовательской функции имеет вид

```
typedef void MPI_User_function(void* invec, void* inoutvec,  
                                int *len, MPI_Datatype *datatype);
```

Пусть $u[0], \dots, u[len-1]$ – элементы в коммуникационном буфере, характеризующемся аргументами `invec`, `len` и `datatype` в момент вызова функции; $v[0], \dots, v[len-1]$ – элементы в коммуникационном буфере, характеризующемся аргументами `inoutvec`, `len` и `datatype` в момент вызова функции; $w[0], \dots, w[len-1]$ – элементы в коммуникационном буфере, характеризующемся аргументами `inoutvec`, `len` и `datatype` в момент завершения функции; тогда

$$w[i] = u[i] \circ v[i], \quad i=0, \dots, len-1,$$

где $\circ$ – операция редукции, которую реализует данная функция.

Операция All-Reduce

MPI имеет варианты каждой из операций редукции, где результат возвращается всем процессам группы. MPI требует, чтобы все процессы, участвующие в этих операциях, получили идентичные результаты. Следующая операция отличается от MPI_Reduce тем, что результат появляется в принимающем буфере всех членов группы.

```
int MPI_Allreduce(void* sendbuf, void* recvbuf, int count,  
    MPI_Datatype datatype, MPI_Op op, MPI_Comm comm);
```

Здесь:

sendbuf – начальный адрес посылающего буфера

recvbuf – начальный адрес принимающего буфера

count – количество элементов в посылающем буфере

datatype – тип данных элементов посылающего буфера

op – операция

comm – коммуникатор

Операция Reduce-Scatter

MPI имеет варианты каждой из операций редукции, когда результат рассылается всем процессам в группе в конце операции.

```
int MPI_Reduce_scatter(const void* sendbuf, void* recvbuf,  
    const int recvcunts[], MPI_Datatype datatype,  
    MPI_Op op, MPI_Comm comm);
```

`sendbuf` – начальный адрес посылающего буфера

`recvbuf` – начальный адрес принимающего буфера

`recvcunts` – целочисленный массив, определяющий количество элементов результата, распределенных каждому процессу. Массив должен быть идентичен во всех вызывающих процессах

`datatype` – тип данных элементов буфера ввода (дескриптор)

`op` – операция; `comm` – коммуникатор

Функция сначала производит поэлементную редукцию векторов из

`count =  $\sum_i \text{recvcunts}[i]$` элементов в посылающих буферах, определенных `sendbuf`, `count` и `datatype`. Далее полученный вектор результатов разделяется на `n` непересекающихся сегментов, где `n` – число членов в группе. Сегмент `i` содержит `recvcunt[i]` элементов. `i`-й сегмент посылается `i`-му процессу и хранится в приемном буфере, определяемом `recvbuf`, `recvcunts[i]` и `datatype`.

Префиксная редукция данных

Следующая операция выполняет префиксную редукцию данных, распределенных в группе процессов.

```
int MPI_Scan(void* sendbuf, void* recvbuf, int count,  
             MPI_Datatype datatype, MPI_Op op, MPI_Comm comm );
```

Здесь:

`sendbuf` – начальный адрес посылающего буфера

`recvbuf` – начальный адрес принимающего буфера

`count` – количество элементов в принимающем буфере

`datatype` – тип данных элементов в принимающем буфере

`op` – операция

`comm` – коммуникатор

Функция `MPI_SCAN` используется, чтобы выполнить префиксную редукцию данных, распределенных в группе. Операция возвращает в приемный буфер процесса `i` редукцию значений в посылающих буферах процессов с номерами `0, ..., i` (включительно). Тип поддерживаемых операций, их семантика и ограничения на буфера отправки и приема – такие же, как и для `MPI_REDUCE`.

В новых версиях стандарта MPI (3.0 и более поздних) предусматривается наличие неблокирующих функций для операций коллективного обмена информации, аналогичных рассмотренным ранее блокирующим функциям:

```
int MPI_Ibarrier(MPI_Comm comm, MPI_Request *request);
int MPI_Ibcast(void* buffer, int count, MPI_Datatype datatype,
               int root, MPI_Comm comm, MPI_Request *request);
int MPI_Igather(const void* sendbuf, int sendcount,
               MPI_Datatype sendtype, void* recvbuf, int recvcount,
               MPI_Datatype recvtype, int root, MPI_Comm comm,
               MPI_Request *request);
int MPI_Iscatter(const void* sendbuf, int sendcount,
               MPI_Datatype sendtype, void* recvbuf, int recvcount,
               MPI_Datatype recvtype, int root, MPI_Comm comm,
               MPI_Request *request);
MPI_Iallgather(const void* sendbuf, int sendcount,
               MPI_Datatype sendtype, void* recvbuf, int recvcount,
               MPI_Datatype recvtype, MPI_Comm comm,
               MPI_Request *request);
```

```
int MPI_Ialltoall(const void* sendbuf, int sendcount,  
    MPI_Datatype sendtype, void* recvbuf, int recvcount,  
    MPI_Datatype recvtype, MPI_Comm comm,  
    MPI_Request *request);  
int MPI_Ireduce(const void* sendbuf, void* recvbuf, int count,  
    MPI_Datatype datatype, MPI_Op op, int root, MPI_Comm comm,  
    MPI_Request *request);  
int MPI_Iallreduce(const void* sendbuf, void* recvbuf,  
    int count, MPI_Datatype datatype, MPI_Op op, MPI_Comm comm,  
    MPI_Request *request);  
int MPI_Ireduce_scatter(const void* sendbuf, void* recvbuf,  
    const int recvcounts[], MPI_Datatype datatype, MPI_Op op,  
    MPI_Comm comm, MPI_Request *request);  
int MPI_Iscan(const void* sendbuf, void* recvbuf, int count,  
    MPI_Datatype datatype, MPI_Op op, MPI_Comm comm,  
    MPI_Request *request);
```

ГРУППЫ И КОММУНИКАТОРЫ

- Для создания устойчивых параллельных библиотек интерфейс MPI должен обеспечить:
- Возможность создавать безопасное коммуникационное пространство, которое гарантировало бы, что библиотеки могут выполнять обмен, когда им нужно, без конфликтов с обменами, внешними по отношению к данной библиотеке.
- Возможность определять границы области действия коллективных операций, которые позволяли бы библиотекам избегать ненужного запуска невовлеченных процессов.
- Возможность абстрактного обозначения процессов, чтобы библиотеки могли описывать свои обмены в терминах, удобных для их собственных структур данных и алгоритмов.
- Возможность создавать новые пользовательские средства, такие как дополнительные операции для коллективного обмена. Этот механизм должен обеспечить пользователя или создателя библиотеки средствами эффективного расширения состава операций для передачи сообщений.
- Для поддержки библиотек MPI обеспечивает группы процессов (`groups`), виртуальные топологии (`virtual topologies`), коммутаторы (`communicators`).

- **Коммуникаторы** создают область для всех операций обмена в MPI. Коммуникаторы разделяются на два вида: интра-коммуникаторы (внутригрупповые коммуникаторы), предназначенные для операций в пределах отдельной группы процессов, и интер-коммуникаторы (межгрупповые коммуникаторы), предназначенные для обменов между двумя группами процессов.
- **Группы.** Группы определяют упорядоченную выборку процессов по именам. Таким образом, группы определяют область для парных и коллективных обменов. В MPI группы могут управляться отдельно от коммуникаторов, но в операциях обмена могут использоваться только коммуникаторы.
- **Виртуальная топология** определяет специальное отображение номеров процессов в группе на определенную топологию, и наоборот. Чтобы обеспечить эту возможность, для коммуникаторов предусмотрены специальные конструкторы.

Базовые концепции

- Группа есть упорядоченный набор идентификаторов процессов; каждый процесс в группе связан с целочисленным номером. Нумерация является непрерывной и начинается с нуля. Группы представлены скрытыми объектами группы и, следовательно, не могут быть непосредственно переданы от одного процесса к другому. Группа используется в пределах коммуникатора для описания участников коммуникационной области и ранжирования этих участников путем предоставления им уникальных номеров.
- Имеется предопределенная группа: `MPI_GROUP_EMPTY`, которая является группой без членов. Предопределенная константа `MPI_GROUP_NULL` является значением, используемым для ошибочных дескрипторов группы.
- Контекст есть свойство коммуникаторов, которое позволяет разделять пространство обмена. Сообщение, посланное в одном контексте, не может быть получено в другом контексте. Более того, где это разрешено, коллективные операции независимы от ждущих операций парного обмена. Контексты не являются явными объектами MPI; они проявляются только как часть реализации коммуникаторов.
- Интра-коммуникаторы объединяют концепции группы и контекста для поддержки реализационно-зависимых оптимизаций и прикладных топологий.

- Операции обмена в MPI используют коммутаторы для определения области, в которой должны выполняться парная или коллективная операции.
- Каждый коммутатор содержит группу участников; эта группа всегда участвует в локальном процессе. Источник и адресат сообщения определяются номером процесса в пределах этой группы.
- Для коллективной связи интра-коммутатор определяет набор процессов, которые участвуют в коллективной операции (и их порядок, когда это существенно). Таким образом, коммутатор ограничивает "пространственную" область коммуникации и обеспечивает машинно-независимую адресацию процессов их номерами.
- Начальный для всех возможных процессов интра-коммутатор `MPI_COMM_WORLD` создается сразу при обращении к функции `MPI_Init`. Кроме того, существует коммутатор, который содержит только свой собственный процесс – `MPI_COMM_SELF`.
- Предопределенная константа `MPI_COMM_NULL` есть значение, используемое для неверных дескрипторов коммутатора.
- В реализации MPI со статической моделью обработки коммутатор `MPI_COMM_WORLD` имеет одинаковое значение во всех процессах.

- В реализации MPI, где процессы могут порождаться динамически, возможен случай, когда процесс начинает вычисления, не имея доступа ко всем другим процессам. В таких ситуациях, коммунитор MPI_COMM_WORLD является коммунитором, включающим все процессы, с которыми подключающийся процесс может немедленно связаться, и может одновременно иметь различные значения в различных процессах.
- Все реализации MPI должны обеспечить наличие коммунитора MPI_COMM_WORLD. Он не может быть удален в течение времени существования процесса. Группа, соответствующая этому коммунитору, не появляется как предопределенная константа, но к ней можно обращаться, используя MPI_comm_group.
- MPI не определяет соответствия между номером процесса в MPI_COMM_WORLD и его абсолютным адресом.

Управление группой

Операции управления являются локальными, и их выполнение не требует межпроцессного обмена. Функция

```
int MPI_Group_size(MPI_Group group, int *size);
```

позволяет определить размер группы.

Функция

```
int MPI_Group_rank(MPI_Group group, int *rank);
```

служит для определения номера процесса в группе.

Следующая функция позволяет определить относительную нумерацию процессов в двух различных группах.

```
int MPI_Group_translate_ranks(MPI_Group group1, int n,  
    const int ranks1[], MPI_Group group2, int ranks2[]);
```

group1 – группа1

n – число номеров в массивах ranks1 и ranks2

ranks1 – массив из нуля или более правильных номеров в группе1

group2 – группа2

ranks2 – массив соответствующих номеров в группе2, или MPI_UNDEFINED, если соответствие отсутствует.

Например, если известны номера некоторых процессов в `MPI_COMM_WORLD`, то можно узнать их номера в подмножестве некоторой группы.

Функция

```
int MPI_Group_compare(MPI_Group group1, MPI_Group group2,  
                      int *result);
```

вырабатывается результат `MPI_IDENT`, если члены группы и их порядок в обеих группах совершенно одинаковы. Это происходит, например, если `group1` и `group2` имеют тот же самый дескриптор.

Если члены группы одинаковы, но порядок различен, то вырабатывается результат `MPI_SIMILAR`.

В остальных случаях получается значение `MPI_UNEQUAL`.

Конструкторы групп

Конструкторы групп создают новые группы на основе существующих групп. Данные операции являются локальными, и различные группы могут быть определены в различных процессах. MPI не имеет механизма для формирования группы с нуля – группа может формироваться только на основе другой, предварительно определенной группы. Базовая группа, на основе которой определены все другие группы, является группой, связанной с коммуникатором `MPI_COMM_WORLD`.

Следующая функция возвращает дескриптор группы из соответствующего коммуникатора

```
int MPI_Comm_group(MPI_Comm comm, MPI_Group *group);
```

Группа, соответствующая объединению множеств процессоров из первой и второй группы, возвращается при помощи вызова

```
int MPI_Group_union(MPI_Group group1, MPI_Group group2,  
                   MPI_Group *newgroup);
```

`group1` – первая группа; `group2` – вторая группа;

`newgroup` – объединенная группа (дескриптор)

Сначала в новой группе следуют процессы из первой группы, а затем – процессы из второй группы, не входящие в первую

Группа, соответствующая пересечению множеств процессоров из первой и второй группы, и упорядоченных, как в первой группе, возвращается при помощи вызова

```
int MPI_Group_intersection(MPI_Group group1, MPI_Group group2,  
                           MPI_Group *newgroup);
```

Здесь

group1 – первая группа;

group2 – вторая группа;

newgroup – результирующая группа.

Группа, соответствующая теоретико-множественной разности множеств процессоров из первой и второй групп, возвращается при помощи вызова

```
int MPI_Group_difference(MPI_Group group1, MPI_Group group2,  
                         MPI_Group *newgroup);
```

Здесь

group1 – первая группа;

group2 – вторая группа;

newgroup – результирующая группа.

Функция

```
int MPI_Group_incl(MPI_Group group, int n, const int ranks[],  
                  MPI_Group *newgroup);
```

где

`group` – группа;

`n` – количество элементов в массиве номеров (и размер `newgroup`);

`ranks` – номера процессов в `group`, перешедших в новую группу (массив целых)

`newgroup` – новая группа, полученная из прежней, упорядоченная согласно `ranks`;

создает группу `newgroup`, которая состоит из `n` процессов из `group` с номерами `rank[0], ..., rank[n-1]`; процесс с номером `i` в `newgroup` есть процесс с номером `ranks[i]` в `group`.

Каждый из `n` элементов `ranks` должен быть правильным номером в `group`, и все элементы должны быть различными, иначе программа будет неверна.

Если `n=0`, то `newgroup` имеет значение `MPI_GROUP_EMPTY`.

Эта функция может, например, использоваться, чтобы переупорядочить элементы группы.

Функция

```
int MPI_Group_excl(MPI_Group group, int n, const int ranks[],  
                  MPI_Group *newgroup);
```

где

`group` – группа;

`n` – количество элементов в массиве номеров;

`ranks` – массив целочисленных номеров в `group`, не входящих в `newgroup`;

`newgroup` – новая группа, полученная из прежней, сохраняющая порядок, определенный `group` (дескриптор)

позволяет исключить `n` процессов из создаваемой группы `newgroup`.

Именно, создается группа `newgroup`, которая получена путем удаления из `group` процессов с номерами `ranks[0], ..., ranks[n-1]`. Упорядочивание процессов в `newgroup` идентично упорядочиванию в `group`. Каждый из `n` элементов `ranks` должен быть правильным номером в `group`, и все элементы должны быть различными; в противном случае программа неверна. Если `n=0`, то `newgroup` идентична `group`.

Функция

```
int MPI_Group_free(MPI_Group *group);
```

маркирует объект группы для удаления. Дескриптор `group` устанавливается вызовом в состояние `MPI_GROUP_NULL`.

Любая выполняющаяся операция, использующая эту группу, завершится нормально.

Управление коммутаторами

Доступ к коммутаторам

Все следующие операции являются локальными.

Возврат числа процессоров `size` в коммутаторе `comm`

```
int MPI_Comm_size(MPI_Comm comm, int *size);
```

Возврат номера (т.е. ранга `rank`) процесса в частной группе коммутатора `comm`

```
int MPI_Comm_rank(MPI_Comm comm, int *rank);
```

Сравнение коммутаторов реализуется функцией

```
int MPI_Comm_compare(MPI_Comm comm1, MPI_Comm comm2,  
int *result);
```

Результат `MPI_IDENT` появляется тогда и только тогда, когда `comm1` и `comm2` являются дескрипторами для одного и того же объекта. Результат `MPI_CONGRUENT` появляется, если исходные группы идентичны по компонентам и нумерации; эти коммутаторы отличаются только контекстом. Результат `MPI_SIMILAR` имеет место, если члены группы обоих коммутаторов являются одинаковыми, но порядок их нумерации различен. В противном случае выдается результат `MPI_UNEQUAL`.

Конструкторы коммутаторов

Нижеперечисленные функции являются коллективными и вызываются всеми процессами в группе, связанной с `comm`. В MPI для создания нового коммутатора необходим исходный коммутатор. Основным коммутатором для всех MPI коммутаторов является коммутатор `MPI_COMM_WORLD`.

- Следующая функция дублирует существующий коммутатор `comm` и возвращает в аргументе `newcomm` новый коммутатор с той же группой:

```
int MPI_Comm_dup(MPI_Comm comm, MPI_Comm *newcomm);
```

- Создание нового коммутатора `newcomm` с коммуникационной группой, определенной аргументом `group`, и новым контекстом, обеспечивается функцией

```
int MPI_Comm_create(MPI_Comm comm, MPI_Group group, MPI_Comm *newcomm);
```

Функция возвращает `MPI_COMM_NULL` для процессов, не входящих в `group`. Запрос неверен, если не все аргументы в `group` имеют одинаковое значение или если `group` не является подмножеством группы, связанной с `comm`. Заметим, что запрос должен быть выполнен всеми процессами в `comm`, даже если они не принадлежат новой группе.

```
int MPI_Comm_split(MPI_Comm comm, int color, int key,  
                   MPI_Comm *newcomm);
```

Эта функция делит группу, связанную с `comm`, на непересекающиеся подгруппы по одной для каждого значения `color`. Каждая подгруппа содержит все процессы одного цвета. В пределах каждой подгруппы процессы пронумерованы в порядке, определенном значением аргумента `key`, со связями, разделенными согласно их номеру в старой группе. Для каждой подгруппы создается новый коммуникатор и возвращается в аргументе `newcomm`. Процесс может иметь значение цвета `MPI_UNDEFINED`, тогда переменная `newcomm` возвращает значение `MPI_COMM_NULL`. Это коллективная операция, но каждому процессу разрешается иметь различные значения для `color` и `key`.

Деструктор коммуникатора

```
int MPI_Comm_free(MPI_Comm *comm);
```

Эта коллективная операция маркирует коммуникационный объект для удаления. Дескриптор устанавливается в `MPI_COMM_NULL`. Любые ждущие операции, которые используют этот коммуникатор, будут завершаться нормально; объект фактически удаляется только в том случае, если не имеется никаких других активных ссылок на него.

Виртуальные топологии

- Топология является необязательным атрибутом, который дополняет систему интра-коммуникаторов и не применяется в интер-коммуникаторах.
- Топология обеспечивает удобный способ обозначения процессов в группе (внутри коммуникатора) и оказывает помощь исполнительной системе при размещении процессов в аппаратной среде.
- Во многих параллельных приложениях линейное распределение номеров не отражает в полной мере логической структуры обменов между процессами, которая зависит от структуры задачи и ее математического алгоритма.
- Часто процессы описываются в двух- и трехмерных топологических средах.
- Наиболее общим видом топологии является организация процессов, описываемая графовой структурой.

Конструктор декартовой топологии

```
int MPI_Cart_create(MPI_Comm comm_old, int ndims, int *dims,  
    int *periods, int reorder, MPI_Comm *comm_cart);
```

Создает новый коммуникатор, к которому подключается топологическая информация. Здесь:

`comm_old` – исходный коммуникатор (дескриптор)

`ndims` – размерность создаваемой декартовой решетки (целое)

`dims` – целочисленный массив размера `ndims`, хранящий количество процессов по каждой координате

`periods` – массив логических элементов размера `ndims`, определяющий, периодична (`true`) или нет (`false`) решетка в каждой размерности

`reorder` – нумерация может быть сохранена (`false`) или переупорядочена (`true`) (логическое значение)

`comm_cart` – коммуникатор новой декартовой топологии (дескриптор)

Если полная размерность декартовой решетки меньше, чем размер группы коммуникаторов, то некоторые процессы возвращаются с результатом `MPI_COMM_NULL` по аналогии с `MPI_COMM_SPLIT`. Вызов будет неверным, если он задает решетку большего размера, чем размер группы.

Функция

```
int MPI_Dims_create(int nnodes, int ndims, int *dims);
```

где

`nnodes` – количество узлов решетки,

`ndims` – число размерностей декартовой решетки,

`dims` – целочисленный массив размера `ndims`, указывающий количество вершин в каждой размерности,

помогает пользователю выбрать выгодное распределение процессов по каждой координате в зависимости от числа процессов в группе и некоторых ограничений, определенных пользователем.

Конструктор универсальной (графовой) топологии

```
int MPI_Graph_create(MPI_Comm comm_old, int nnodes,  
                    const int index[], const int edges[],  
                    int reorder, MPI_Comm *comm_graph);
```

`comm_old` – входной коммуникатор (дескриптор)

`nnodes` – количество узлов графа (целое)

`index` – массив целочисленных значений, описывающий степени вершин

`edges` – массив целочисленных значений, описывающий ребра графа

`reorder` – номера могут быть переупорядочены (`true`) или нет (`false`)

`comm_graph` – созданный коммуникатор с графовой топологией (дескриптор)

Структуру графа определяют три параметра: `nnodes`, `index` и `edges`. `nnodes` – число узлов графа, пронумерованных от 0 до `nnodes-1`. `j`-ый элемент массива `index` хранит общее число соседей первых `j` вершин графа. Списки соседей вершин 0, 1, ..., `nnodes-1` хранятся в последовательности ячеек массива `edges`. Общее число элементов в `index` есть `nnodes`, а общее число элементов в `edges` равно числу ребер графа. Если количество узлов графа `nnodes` меньше, чем размер группы коммуникатора, то некоторые процессы получают значение `MPI_COMM_NULL`. Нельзя определить граф большего размера, чем размер группы исходного коммуникатора.

Топологические функции запроса

`int MPI_Topo_test(MPI_Comm comm, int *status);`

возвращает тип топологии, переданной коммуникатору. Выходное значение `status` имеет одно из следующих значений:

`MPI_GRAPH` – топология графа,

`MPI_CART` – декартова топология,

`MPI_UNDEFINED` – топология не определена.

`int MPI_Graphdims_get(MPI_Comm comm, int *nnodes, int *nedges);`

Возвращает число вершин и ребер в графе. Здесь:

`comm` – коммуникатор группы с графовой топологией

`nnodes` – число вершин графа; `nedges` – число ребер графа

`int MPI_Graph_get(MPI_Comm comm, int maxindex, int maxedges,
int *index, int *edges);`

Возвращает информацию о структуре графа. Здесь:

`comm` коммуникатор с графовой топологией

`maxindex` – длина вектора `index`,

`maxedges` – длина вектора `edges`,

`index` – целочисленный массив, содержащий структуру графа

`edges` – целочисленный массив, содержащий структуру графа


```
int MPI_Cartdim_get(MPI_Comm comm, int *ndims);
```

– возвращает число измерений `ndims` декартовой топологии, связанной с коммуникатором `comm`

Более подробная информация о декартовой топологии возвращается функцией

```
int MPI_Cart_get(MPI_Comm comm, int maxdims, int dims[],  
                 int periods[], int coords[]);
```

Здесь

`comm` – коммуникатор с декартовой топологией

`maxdims` – длина векторов `dims`, `periods` и `coords`

`dims` – число процессов по каждой декартовой размерности (целочисленный массив)

`periods` – периодичность (`true/false`) для каждой декартовой размерности (массив логических элементов)

`coords` – координаты вызываемых процессов в декартовой системе координат (целочисленный массив)

Для группы процессов с декартовой структурой следующая функция переводит логические координаты процессов в номера, которые используются в процедурах парного обмена:

```
int MPI_Cart_rank(MPI_Comm comm, const int coords[],  
                  int *rank);
```

`comm` – коммуникатор с декартовой топологией;

`coords` – целочисленный массив (размера `ndims`), описывающий декартовы координаты процесса;

`rank` – номер указанного процесса.

Перевод номера процесса в его логические координаты в декартовой топологии:

```
int MPI_Cart_coords(MPI_Comm comm, int rank, int maxdims,  
                    int coords[]);
```

Здесь:

`comm` – коммуникатор с декартовой топологией;

`rank` – номер процесса внутри группы `comm`;

`maxdims` – длина вектора `coord`;

`coords` – целочисленный массив (размера `ndims`), содержащий декартовы координаты указанного процесса.

Число процессов, соседних процессу с нужным номером в коммуникаторе с топологией графа, возвращается функцией

```
int MPI_Graph_neighbors_count(MPI_Comm comm, int rank,  
                             int *nneighbors);
```

Здесь:

`comm` – коммуникатор с графовой топологией;

`rank` – номер процесса в группе `comm`;

`nneighbors` – число процессов, являющихся соседними указанному процессу.

Номера процессов, соседних процессу с нужным номером в коммуникаторе с топологией графа, возвращается функцией

```
int MPI_Graph_neighbors(MPI_Comm comm, int rank,  
                        int maxneighbors, int neighbors[]);
```

Здесь:

`comm` – коммуникатор с графовой топологией;

`rank` – номер процесса в группе `comm`;

`maxneighbors` – размер массива `neighbors`;

`neighbors` – номера процессов, соседних данному (целочисленный массив).

Сдвиг в декартовых координатах

Если используется декартова топология, то операцию `MPI_Sendrecv` можно выполнить путем сдвига данных вдоль направления координаты. В качестве входного параметра `MPI_Sendrecv` использует номер процесса-отправителя для приема и номер процесса-получателя – для передачи. Если функция `MPI_Cart_shift` выполняется для декартовой группы процессов, то она передает вызывающему процессу эти номера, которые затем могут быть использованы для `MPI_Sendrecv`. Пользователь определяет направление координаты и величину шага (положительный или отрицательный). Эта функция является локальной.

Здесь:

```
int MPI_Cart_shift(MPI_Comm comm, int direction, int disp, int
*rank_source, int *rank_dest);
```

`comm` – коммуникатор с декартовой топологией

`direction` – координата сдвига (целое)

`disp` – смещение (> 0 : смещение вверх, < 0 : смещение вниз)

`rank_source` – номер процесса-отправителя (целое)

`rank_dest` – номер процесса-получателя (целое)

Декомпозиция декартовых структур

Если декартова топология была создана с помощью функции `MPI_Cart_create`, то функция `MPI_Cart_sub` может использоваться для декомпозиции группы в подгруппы, которые являются декартовыми подрешетками, и построения для каждой подгруппы коммуникаторов с подрешеткой с декартовой топологией.

```
int MPI_Cart_sub(MPI_Comm comm, const int remain_dims[],  
                 MPI_Comm *newcomm);
```

Здесь

`comm` – коммуникатор с декартовой топологией;

`remain_dims` – `j`-й элемент в `remain_dims` показывает, содержится ли `j`-я размерность в подрешетке (`true`) или нет (`false`) (вектор логических элементов);

`newcomm` – коммуникатор, содержащий подрешетку, которая включает вызываемый процесс.

Новые возможности MPI 3.0

Новый стандарт MPI версии 3.0 определяет ряд функций для коллективного обмена информацией между процессами, соседними в топологии, связанной с коммуникатором. Например, сбор данных по всем соседним процессам выполняется функцией

```
int MPI_Neighbor_allgather(const void* sendbuf, int sendcount,  
    MPI_Datatype sendtype, void* recvbuf, int recvcount,  
    MPI_Datatype recvtype, MPI_Comm comm);
```

Здесь:

`sendbuf` – адрес первого байта буфера, из которого отсылаются данные;

`sendcount` – число элементов, отсылаемых каждому из соседних процессов;

`sendtype` – тип элементов отсылаемых данных;

`recvbuf` – адрес первого байта буфера для приема информации;

`recvcount` – число элементов, принимаемых из каждого соседнего процесса;

`recvtype` – тип элементов принимаемых данных

`comm` – коммуникатор с заданной топологией